

Taller Práctico: Relacionando la Matemática de TF2 con Turtlesim (ROS 2 Jazzy)

En este taller utilizaremos el simulador `turtlesim` para dar vida a los conceptos matemáticos (matrices de rotación, traslación, y grupos $SE(2)$) vistos en el cuaderno de transformaciones. El objetivo es que logres conectar las ecuaciones matemáticas con los comandos y herramientas reales de ROS 2.

Objetivos del Taller

1. Relacionar la matriz de transformación homogénea (T) con la salida en tiempo real de los frames en ROS 2.
2. Comprender cómo ROS 2 maneja internamente las rotaciones usando cuaterniones en lugar de matrices de rotación simples.
3. Comprobar empíricamente la **composición de transformaciones** ($T_A^C = T_A^B \cdot T_B^C$) utilizando `tf2_echo` y el simulador.

Requisitos Previos

Abre una terminal, asegúrate de haber inicializado ROS 2 Jazzy (`source /opt/ros/jazzy/setup.bash`) y verifica que los paquetes estén instalados:

```
sudo apt install ros-jazzy-turtle-tf2-py ros-jazzy-tf2-tools ros-jazzy-turtlesim
```

Actividad 1: Mapeando la Matriz Homogénea al Simulador

Sabemos que una transformación en 2D del frame de la tortuga (`turtle1`) al frame del mundo (`world`) se define mediante la matriz:

$$T_{\text{world}}^{\text{turtle1}} = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix}$$

Vamos a ver de dónde salen estos valores de x , y , y θ en la práctica.

1. **Terminal 1:** Ejecuta la demostración de TF2. Esto creará el entorno con dos tortugas y sus respectivos publicadores de transformaciones.

```
ros2 launch turtle_tf2_py turtle_tf2_demo.launch.py
```

2. **Terminal 2:** Habilita el control por teclado para mover a `turtle1`.

```
ros2 run turtlesim turtle_teleop_key
```

3. **Terminal 3:** “Escucha” la matriz de transformación en tiempo real con este comando:

```
ros2 run tf2_ros tf2_echo world turtle1
```

Análisis Matemático: Observa la salida en la terminal. Verás algo como esto: - **Translation:** `[x, y, 0.000]` -> Estos son los valores exactos del vector de traslación $\mathbf{t} = [x, y]^T$ de nuestra matriz. (En `turtlesim`, Z es 0). - **Rotation: in Quaternion** `[x: 0.000, y: 0.000, z: 0.382, w: 0.923]` -> ROS 2 no muestra la matriz de rotación 2×2 directamente para ahorrar cálculos, sino que usa **cuaterniones**. - *Reto matemático:* El ángulo de giro de la tortuga (Yaw, θ) se esconde en el cuaternión. Para un giro en 2D, el ángulo θ se calcula como: $\theta = 2 \cdot \text{atan2}(z, w)$. Si usas Python para calcularlo con los números que te arroja la terminal, obtendrás exactamente el ángulo θ que necesitas para armar tu matriz T .

Actividad 2: Composición de Transformaciones y el Árbol TF

Si tenemos dos tortugas (`turtle1` y `turtle2`), la posición de ambas se reporta siempre con respecto a `world`. En código matemático tenemos T_W^{T1} y T_W^{T2} .

¿Qué pasa si queremos saber la posición de `turtle2` desde la perspectiva de `turtle1`? Matemáticamente calculamos la inversa y multiplicamos:

$$T_{T1}^{T2} = (T_W^{T1})^{-1} \cdot T_W^{T2}$$

¡ROS 2 hace esta multiplicación matricial por nosotros a través del Árbol TF!

1. En la **Terminal 3**, visualiza el árbol matemático que ROS ha construido:

```
ros2 run tf2_tools view_frames
```

Abre el archivo `frames.pdf` generado. Verás que `world` es el nodo padre, y de él se derivan las dos transformaciones matriciales iniciales. ROS usa este grafo para calcular cualquier multiplicación intermedia.

2. Pídele a ROS 2 que resuelva la ecuación T_{T1}^{T2} imprimiendo el resultado:

```
ros2 run tf2_ros tf2_echo turtle1 turtle2
```

3. **Ejercicio de comprobación:**

- Detén las tortugas (no uses el teclado).
- Anota los valores de traslación de `world` -> `turtle1`.
- Anota los valores de traslación de `world` -> `turtle2`.
- Reemplaza esos valores en la celda número 14 de tu Jupyter Notebook (Ejemplo con dos tortugas) para calcular teóricamente T_{T1}^{T2} en Python usando `np.linalg.inv()`.
- Compara la traslación resultante en Python con la que te está imprimiendo `tf2_echo turtle1 turtle2`. ¡Deberían ser idénticas!

Actividad 3: Geometría Visual en RViz2

Visualizar los sistemas de referencia y los puntos de coordenadas permite entender qué significa multiplicar un punto por una matriz: $\mathbf{p}_{world} = T_{world}^{turtle1} \cdot \mathbf{p}_{turtle1}$.

1. Abre **RViz2** desde una terminal:

```
rviz2
```

2. Configura la vista:

- En **Global Options** -> **Fixed Frame**, selecciona `world`. El centro de RViz (0,0) será ahora el origen de nuestra matriz identidad I .
- Clic en **Add** (abajo izquierda) -> selecciona **TF**.

3. Ahora puedes ver cómo las matrices T_W^{T1} y T_W^{T2} están representadas por los ejes RGB (Rojo=X, Verde=Y).

4. Gira la `turtle1` en el simulador. Notarás que el eje Rojo (X) del frame `turtle1` en RViz siempre apunta hacia donde mira la tortuga. Si multiplicaras el vector local de la tortuga $\mathbf{p} = [1, 0, 1]^T$ por su matriz actual T , el punto resultante en RViz caería exactamente 1 metro adelante de ella a lo largo del eje rojo.

Ejercicios Finales de Análisis

1. ¿Por qué al solicitar `tf2_echo turtle1 turtle2` los valores cambian constantemente, incluso si `turtle2` se queda quieta, pero tú rotas a `turtle1` sobre su propio eje? Explícalo usando las propiedades de la matriz de rotación R .
2. De acuerdo con lo observado en RViz2 y la regla matemática de homogeneización, ¿qué sucede con la coordenada Z de la traslación en el caso estricto de turtlesim y por qué?
3. Convierte manualmente un cuaternión dado por `tf2_echo world turtle1` a un ángulo θ en radianes (puedes usar la fórmula del Actividad 1), y construye la matriz de rotación de 2×2 .

Desafío Final: Agregando un nuevo Frame (El Sensor)

Imagina que le instalamos un sensor de visión a `turtle1`. Físicamente, el sensor está montado mecánicamente exactamente **0.5 metros hacia adelante** de la tortuga (es decir, una traslación de +0.5 sobre el eje X local del frame `turtle1`).

Tu misión es decirle a ROS 2 que existe este nuevo frame llamado `sensor` y relacionarlo matemáticamente con `turtle1`, para luego visualizar cómo se mueve por el mapa.

Pasos del desafío: 1. Abre una nueva terminal y usa la herramienta `static_transform_publisher` para publicar esta matriz constante T_{T1}^{sensor} . Para ROS 2 Jazzy, se recomienda el uso de **argumentos nombrados** para evitar la clásica confusión con el orden de las rotaciones de Euler: `bash ros2 run tf2_ros static_transform_publisher --x 0.5 --y 0.0 --z`

0.0 --roll 0.0 --pitch 0.0 --yaw 0.0 --frame-id turtle1 --child-frame-id sensor 2. Genera nuevamente el árbol de frames en PDF usando `view_frames`. ¿Cómo se ve la jerarquía ahora? 3. En **RViz2**, ve al panel lateral, busca las opciones de “TF” y asegúrate de que el nuevo frame **sensor** esté habilitado. 4. Conduce la tortuga con el teclado y observa cómo el frame del sensor gira y se mueve solidariamente con ella, respetando siempre la distancia de 0.5 metros. 5. **Reto matemático:** Usa el comando `ros2 run tf2_ros tf2_echo world sensor` mientras la tortuga está detenida apuntando en diagonal (ej. $\theta = 45^\circ$). Usa la ecuación vista en tu cuaderno $\mathbf{p}_{world} = T_{world}^{turtle1} \cdot \mathbf{p}_{sensor}$ para explicar por qué la posición X, Y del sensor en el mundo no se calcula simplemente sumando 0.5 a la posición X, Y de la tortuga. ¡Demuestra que la rotación de la matriz afecta la traslación final!